Building Multilayer Perceptron Models in PyTorch

by **Adrian Tam** on April 8, 2023 in **Deep Learning with PyTorch** 2

The PyTorch library is for deep learning. Deep learning, indeed, is just another name for a large-scale neural network or multilayer perceptron network. In its simplest form, multilayer perceptrons are a sequence of layers connected in tandem. In this post, you will discover the simple components you can use to create neural networks and simple deep learning models in PyTorch.

Kick-start your project with my book Deep Learning with PyTorch. It provides **self-study tutorials** with **working code**.

Let's get started.

Building multilayer perceptron models in PyTorch
Photo by Sharon Cho. Some rights reserved.

Overview

This post is in six parts; they are:

- Neural Network Models in PyTorch
- Model Inputs
- Layers, Activations, and Layer Properties
- Loss Functions and Model Optimizers
- Model Training and Inference
- Examination of a Model

Neural Network Models in PyTorch

PyTorch can do a lot of things, but the most common use case is to build a deep learning model. The simplest model can be defined using `Sequential` class, which is just a linear stack of layers connected in tandem. You can create a `Sequential` model and define all the layers in one shot; for example:

```
import torch
import torch.nn as nn

model = nn.Sequential(...)
```

You should have all your layers defined inside the parentheses in the processing order from input to output. For example:

```
model = nn.Sequential(
    nn.Linear(764, 100),
    nn.ReLU(),
    nn.Linear(100, 50),
    nn.ReLU(),
    nn.Linear(50, 10),
    nn.Sigmoid()
)
```

The other way of using `Sequential` is to pass in an ordered dictionary in which you can assign names to each layer:

```
from collections import OrderedDict
import torch.nn as nn

model = nn.Sequential(OrderedDict([
    ('dense1', nn.Linear(764, 100)),
    ('act1', nn.ReLU()),
    ('dense2', nn.Linear(100, 50)),
    ('act2', nn.ReLU()),
    ('output', nn.Linear(50, 10)),
    ('outact', nn.Sigmoid()),
]))
```

And if you would like to build the layers one by one instead of doing everything in one shot, you can do the following:

```
model = nn.Sequential()
model.add_module("dense1", nn.Linear(8, 12))
model.add_module("act1", nn.ReLU())
model.add_module("dense2", nn.Linear(12, 8))
model.add_module("act2", nn.ReLU())
model.add_module("output", nn.Linear(8, 1))
model.add_module("outact", nn.Sigmoid())
```

You will find this helpful in a more complex case where you need to build a model based on some conditions.

Model Inputs

The first layer in your model hints at the shape of the input. In the example above, you have `nn.Linear(764, 100)` as the first layer. Depending on the different layer type you use, the arguments may bear different meanings. But in this case, it is a `Linear` layer (also known as a dense layer or fully connected layer), and the two arguments tell the input and output dimensions of **this layer**.

Note that the size of a batch is implicit. In this example, you should pass in a PyTorch tensor of shape `(n, 764)` into this layer and expect a tensor of shape `(n, 100)` in return, where `n` is the size of a batch.

Layers, Activations, and Layer Properties

There are many kinds of neural network layers defined in PyTorch. In fact, it is easy to define your own layer if you want to. Below are some common layers that you may see often:

- `nn.Linear(input, output)`: The fully-connected layer
- `nn.Conv2d(in_channel, out_channel, kernel_size)`: The 2D convolution layer, popular in image processing networks
- `nn.Dropout(probability)`: Dropout layer, usually added to a network to introduce regularization
- `nn.Flatten()`: Reshape a high-dimensional input tensor into 1-dimensional (per each sample in a batch)

Besides layers, there are also activation functions. These are functions applied to each element of a tensor. Usually, you take the output of a layer and apply the activation before feeding it as input to a subsequent layer. Some common activation functions are:

- `nn.ReLU()`: Rectified linear unit, the most common activation nowadays
- `nn.Sigmoid()` and `nn.Tanh()`: Sigmoid and hyperbolic tangent functions, which are the usual choice in older literature
- `nn.Softmax()`: To convert a vector into probability-like values; popular in classification networks

You can find a list of all the different layers and activation functions in PyTorch's documentation.

The design of PyTorch is very modular. Therefore, you don't have much to adjust in each component. Take this `Linear` layer as an example. You can only specify the input and output shape but not other details, such as how to initialize the weights. However, almost all the components can take two additional arguments: the device and the data type.

A PyTorch device specifies where this layer will execute. Normally, you choose between the CPU and the GPU or omit it and let PyTorch decide. To specify a device, you do the following (CUDA means a supported nVidia GPU):

```
nn.Linear(764, 100, device="cpu")
```

or

```
nn.Linear(764, 100, device="cuda:0")
```

The data type argument (`dtype`) specifies what kind of data type this layer should operate on. Usually, it is a 32-bit float, and usually, you don't want to change that. But if you need to specify a different type, you must do so using PyTorch types, e.g.,

```
nn.Linear(764, 100, dtype=torch.float16)
```

Loss Function and Model Optimizers

A neural network model is a sequence of matrix operations. The matrices that are independent of the input and kept inside the model are called weights. Training a neural network will **optimize** these weights so that they produce the output you want. In deep learning, the algorithm to optimize these weights is gradient descent.

There are many variations of gradient descent. You can make your choice by preparing an optimizer for your model. It is not part of the model, but you will use it alongside the model during training. The way you use it includes defining a **loss function** and minimizing the loss function using the optimizer. The loss function will give a **distance score** to tell how far away the model's output is from your desired output. It compares the output tensor of the model to the expected tensor, which is called the **label** or the **ground truth** in a different context. Because it is provided as part of the training dataset, a neural network model is a supervised learning model.

In PyTorch, you can simply take the model's output tensor and manipulate it to calculate the loss. But you can also make use of the functions provided in PyTorch for that, e.g.,

```
loss_fn = nn.CrossEntropyLoss()
loss = loss_fn(output, label)
```

In this example, the `loss_fn` is a function, and `loss` is a tensor that supports automatic differentiation. You can trigger the differentiation by calling `loss.backward()`.

Below are some common loss functions in PyTorch:

- `nn.MSELoss()`: Mean square error, useful in regression problems
- `nn.CrossEntropyLoss()`: Cross entropy loss, useful in classification problems
- `nn.BCELoss()`: Binary cross entropy loss, useful in binary classification problems

Creating an optimizer is similar:

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

All optimizers require a list of all parameters that it needs to optimize. It is because the optimizer is created outside the model, and you need to tell it where to look for the parameters (i.e., model weights). Then, the optimizer will take the gradient as computed by the `backward()` function call and apply it to the parameters based on the optimization algorithm.

This is a list of some common optimizers:

- `torch.optim.Adam()`: The Adam algorithm (adaptive moment estimation)
- `torch.optim.NAdam()`: The Adam algorithm with Nesterov momentum
- `torch.optim.SGD()`: Stochastic gradient descent
- `torch.optim.RMSprop()`: The RMSprop algorithm

You can find a list of all provided loss functions and optimizers in PyTorch's documentation. You can learn about the mathematical formula of each optimization algorithm on the respective optimizers' page in the documentation.

Model Training and Inference

PyTorch doesn't have a dedicated function for model training and evaluation. A defined model by itself is like a function. You pass in an input tensor and get back the output tensor. Therefore, it is your responsibility to write the training loop. A minimal training loop is like the following:

```
for n in range(num_epochs):
    y_pred = model(X)
    loss = loss_fn(y_pred, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

If you already have a model, you can simply take `y_pred = model(X)` and use the output tensor `y_pred` for other purposes. That's how you use the model for prediction or inference. A model, however, does not expect one input sample but a batch of input samples in one tensor. If the model is to take an input vector (which is one-dimensional), you should provide a two-dimensional tensor to the model. Usually, in the case of inference, you deliberately create a batch of one sample.

Examination of a Model

Once you have a model, you can check what it is by printing it:

```
print(model)
```

This will give you, for example, the following:

```
Sequential(
  (0): Linear(in_features=8, out_features=12, bias=True)
  (1): ReLU()
  (2): Linear(in_features=12, out_features=8, bias=True)
  (3): ReLU()
  (4): Linear(in_features=8, out_features=1, bias=True)
  (5): Sigmoid()
)
```

If you would like to save the model, you can use the `pickle` library from Python. But you can also access it using PyTorch:

```
torch.save(model, "my_model.pickle")
```

This way, you have the entire model object saved in a pickle file. You can retrieve the model with:

```
model = torch.load("my_model.pickle")
```

But the recommended way of saving a model is to leave the model design in code and keep only the weights. You can do so with:

```
torch.save(model.state_dict(), "my_model.pickle")
```

The `state_dict()` function extracts only the states (i.e., weights in a model). To retrieve it, you need to rebuild the model from scratch and then load the weights like this:

```
model = nn.Sequential(...)
model.load_state_dict(torch.load("my_model.pickle"))
```

Resources

You can learn more about how to create simple neural networks and deep learning models in PyTorch using the following resources:

Online resources

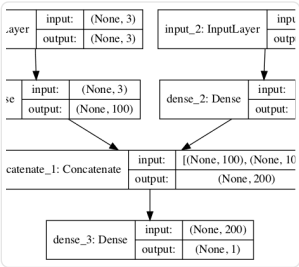
- `torch.nn` documentation
- `torch.optim` documentation
- PyTorch tutorials

Summary

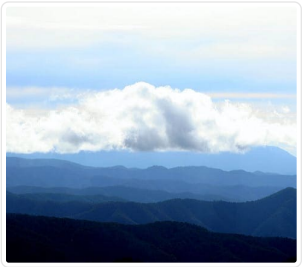
In this post, you discovered the PyTorch API that you can use to create artificial neural networks and deep learning models. Specifically, you learned about the life cycle of a PyToch model, including:

- Constructing a model
- Creating and adding layers and activations
- Preparing a model for training and inference

More On This Topic

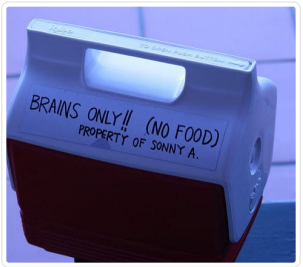


How to Develop Multilayer Perceptron Models for Time…



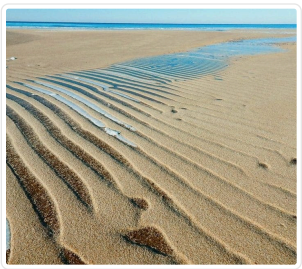
How to Configure Multilayer Perceptron Network for…

How to Build Multi-Layer Perceptron Neural Network…



Crash Course on Multi-Layer Perceptron Neural Networks

How To Implement The Perceptron Algorithm From…



Perceptron Algorithm for Classification in Python

About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.
View all posts by Adrian Tam →

‹ Using Autograd in PyTorch to Solve a Regression Problem

Develop Your First Neural Network with PyTorch, Step by Step ›

2 Responses to *Building Multilayer Perceptron Models in PyTorch*

User October 30, 2023 at 8:51 pm #

REPLY

Shouldn't the loss function be BCELoss() instead of CrossEntropyLoss()? The model outputs a Sigmoid() probability but CrossEntropyLoss() expects raw logits as input

James Carmichael October 31, 2023 at 11:03 am #

REPLY

Hi User…The following resource may be of interest to you:

https://medium.com/dejunhuang/learning-day-57-practical-5-loss-function-crossentropyloss-vs-bceloss-in-pytorch-softmax-vs-bd866c8a0d23

Leave a Reply

Name (required)

Email (will not be published) (required)

SUBMIT COMMENT

Welcome!
I'm *Jason Brownlee* PhD
and I **help developers** get results with **machine learning**.
Read more

Never miss a tutorial:

Picked for you:

PyTorch Tutorial: How to Develop Deep Learning Models with Python


LSTM for Time Series Prediction in PyTorch


Deep Learning with PyTorch (9-Day Mini-Course)


Calculating Derivatives in PyTorch


Building a Regression Model in PyTorch